

CRYPTOGRAPHY AND NETWORK SECURITY

ETTCS801 – Integrated Situation

Securing a Student Records System: Risk Assessment, Encryption, Integrity Checking and Firewall Filtering

Student name	MULUMBA BAKILONGO NICOLAS
Student number	4202670032
Course / Programme	Cryptography and Network Security
Lecturer / Examiner	Isaac TUMWINE
Submission date	28 Sept 2026
GitHub repository	github.com/nicolasmulumba000-create/cryptography-network-security-exam

Table of Contents

1. Introduction	3
2. Risk Assessment	3
2.1 Assets	3
2.2 Vulnerabilities and Consequences	3
2.3 Risk Ranking	3
2.4 Recommended Controls	4
3. File Encryption and Decryption.....	4
3.1 Design.....	4
3.2 Commands	4
3.3 Evidence	5
3.4 Verification.....	5
4. Integrity Checking with SHA-256	5
4.1 Test procedure	5
4.2 Evidence	5
5. Error Handling	6
5.1 Evidence	6
6. Network Traffic Filtering	7
6.1 Network design	7
6.2 Rule set.....	7
6.3 Simulation	7
6.4 Evidence	8
7. Project Structure and Version Control.....	8
8. Conclusion.....	9
Limitations and further work	9
References	9
Appendix A: Evidence Checklist	Error! Bookmark not defined.

1. Introduction

This report documents the work completed for the ETTC801 integrated situation. The scenario concerns a central student-records server that stores academic and personal information and is accessed by authorised staff across campuses. The task is to identify the security risks to this system and to implement practical technical controls that reduce them.

The project covers four areas, each implemented in the accompanying GitHub repository:

- A risk assessment identifying assets, vulnerabilities, consequences, risk ranking and controls.
- A Python security tool that encrypts and decrypts a sample student record using Fernet symmetric encryption.
- SHA-256 integrity checking to detect unauthorised modification of the record.
- A network firewall design and simulation that restricts access to the records server.

All source code, sample data and documentation are available in the repository. Screenshots supplied in this report serve as evidence that each component was executed and tested.

2. Risk Assessment

2.1 Assets

Table 1: Identified assets

Asset	Description
Student records	Academic and personal information about students, stored on the central server. This is the most valuable asset because its exposure affects student privacy.
Student records server	The central server that stores the records and serves them to authorised users (192.168.1.70 in the test environment).
Network services	The services that allow authorised staff and campuses to reach the records system (TCP port 8443).

2.2 Vulnerabilities and Consequences

Table 2: Vulnerabilities and their consequences

Vulnerability	Possible consequence
Weak staff passwords	Unauthorised users may gain access to staff accounts and therefore to student records.
Unencrypted file transfers	Student information may be intercepted while it is transmitted.
Guest network access to the records server	Unauthorised users on the guest network may access or attack the records server.

2.3 Risk Ranking

Each risk was rated for likelihood and impact. Weak passwords are ranked highest because they are the easiest weakness for an attacker to exploit and can give direct access to sensitive data.

Table 3: Risk ranking

Risk	Likelihood	Impact	Reason
Weak staff passwords	High	High	Weak credentials are easier to compromise and may give access to sensitive records.
Unencrypted file transfers	Medium	High	Data sent without encryption may be exposed during transfer.
Guest access to records server	Medium	High	Unauthorised network users could attempt to access the records server.

2.4 Recommended Controls

Table 4: Recommended controls

Risk	Control
Weak staff passwords	Enforce strong password requirements and secure authentication practices (for example, minimum length, complexity and multi-factor authentication).
Unencrypted file transfers	Use encrypted communication for transferring student records. The encryption component of this project demonstrates protecting the record itself.
Guest network access	Apply firewall rules and network segmentation to block guest access to the records server (implemented in Section 6).

3. File Encryption and Decryption

3.1 Design

The tool `security_tool.py` uses the Fernet implementation from the Python cryptography library. Fernet provides authenticated symmetric encryption: it combines AES in CBC mode with a SHA-256 HMAC, so data is both kept confidential and checked for tampering when decrypted. The same key encrypts and decrypts the data.

A key design decision is where the key is stored. The key file (`encryption.key`) is created one folder above the project directory, outside the repository. This prevents the key from being committed to GitHub, which would defeat the purpose of encrypting the record.

3.2 Commands

```
python security_tool.py key      # generate the Fernet key outside the repo
python security_tool.py encrypt  # student_record.txt -> student_record.enc
python security_tool.py decrypt  # student_record.enc ->
student_record_decrypted.txt
```

3.3 Evidence

Figure 1 shows the sample student record before encryption. Figure 2 shows key generation, encryption and decryption being run. Figure 3 shows that the encrypted file is unreadable and that the key file sits outside the repository.

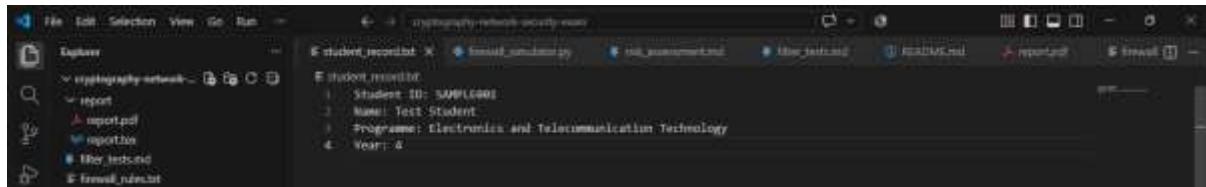


Figure 1: Original student_record.txt (plaintext)



Figure 2: Running key, encrypt and decrypt commands in the terminal

3.4 Verification

After decryption, the tool compares the decrypted bytes with the original file. If they are identical it reports successful verification; otherwise it reports failure. This confirms the encrypt–decrypt cycle is lossless.

4. Integrity Checking with SHA-256

SHA-256 produces a fixed-length digest of a file. Any change to the file, even a single character, produces a completely different digest. The tool stores the digest of the original record as a baseline in integrity_hash.txt and later recalculates it to detect modification. The file is read in 4096-byte chunks so the method also works for large files.

```
python security_tool.py hash      # calculate and save baseline SHA-256
python security_tool.py verify    # compare current SHA-256 with baseline
```

4.1 Test procedure

- Step 1: Run the hash command to save the baseline digest.
- Step 2: Run verify on the unchanged file. The expected result is “Integrity check PASSED”.
- Step 3: Edit student_record.txt (for example, change a grade or a name) and run verify again. The expected result is “Integrity check FAILED”.
- Step 4: Restore the original content and run verify to confirm it passes again.

4.2 Evidence



Figure 4: Baseline hash created and integrity check passing

```

Encrypted file created: student_record.enc
PS C:\Users\Ivan\Downloads\cryptography-network-security-exam> python security_tool.py encrypt
Error: student_record.txt not found.

```

Figure 5: Integrity check failing after the record was modified

Note: because Fernet already authenticates ciphertext, SHA-256 here has a complementary role – it checks the plaintext record at rest, whereas Fernet’s HMAC protects the encrypted file.

5. Error Handling

The tool anticipates common failure conditions and reports them with clear messages instead of crashing with a stack trace. The main function catches `FileNotFoundError`, `OSError` and `ValueError` and returns a non-zero exit code.

Table 5: Error conditions and handling

Condition	Handling	Message shown
Encryption key missing	<code>load_key()</code> raises <code>FileNotFoundError</code> with instructions	Encryption key not found ... generate it with the key command
Input or encrypted file missing	Existence checked before opening	File not found: <path>
Wrong key or corrupted ciphertext	<code>InvalidToken</code> caught and re-raised as <code>ValueError</code>	Decryption failed: invalid key or corrupted data
Key already exists	Existing key is never overwritten	Key already exists – no new key generated
Hash baseline missing	Checked before verification	Integrity baseline not found. Run: hash
Unknown or missing command	Help text displayed, exit code 1	Unknown command / provide one command

Refusing to overwrite an existing key is an important safeguard: replacing the key would make every previously encrypted file permanently unrecoverable.

5.1 Evidence

```

PS C:\Users\Ivan\Downloads\cryptography-network-security-exam> python security_tool.py hello
Error: Invalid command 'hello'.

Cryptography Security Toolkit
-----
(commands):
key      Generate encryption key
encrypt  Encrypt student record
decrypt  Decrypt student record
hash     Calculate SHA-256 hash
verify   Check current file integrity
help     Show this help message
PS C:\Users\Ivan\Downloads\cryptography-network-security-exam>

```

Figure 6: Error handling – missing file or wrong command

6. Network Traffic Filtering

6.1 Network design

Table 6: Firewall test environment

Element	Value
Records server	192.168.1.70
Service	Student records application, TCP port 8443
Staff / local network	192.168.1.0/24
Simulated guest network	192.168.2.0/24
Default policy	DROP

6.2 Rule set

The firewall follows a default-deny approach: anything not explicitly permitted is dropped. This is safer than default-allow because forgotten cases fail closed rather than open.

Table 7: Firewall rules

Rule	Action	Description
1	ACCEPT	Allow established and related connections (return traffic for permitted sessions).
2	DROP	Drop all traffic from the guest network 192.168.2.0/24 to the records server 192.168.1.70.
3	ACCEPT	Accept TCP traffic from the staff network 192.168.1.0/24 to 192.168.1.70 on port 8443.
4	DROP	Drop TCP traffic to port 8443 from any source outside the authorised staff network.
5	DROP	Drop all other inbound traffic by default.

Rule order matters. The guest-network DROP is evaluated before the staff ACCEPT so that a guest address can never match a permissive rule.

6.3 Simulation

Because a real multi-network environment was not available, `firewall_simulator.py` models the rule logic using Python's `ipaddress` module. It evaluates a packet (source IP, destination IP, destination port) and returns ACCEPT or DROP with a reason. The staff network is based on the real local network of the test computer, while the guest network and external host are simulated.

```
python firewall_simulator.py
```

Table 8: Firewall test results

#	Test	Source	Expected	Actual	Status
1	Staff → Records service (8443)	192.168.1.50	ACCEPT	ACCEPT	PASS
2	Guest → Records service (8443)	192.168.2.75	DROP	DROP	PASS
3	External host → Records service (8443)	203.0.113.50	DROP	DROP	PASS

Address 203.0.113.50 belongs to the TEST-NET-3 documentation range, so it is a safe stand-in for an external internet host.

6.4 Evidence



```

Python 3.10.4
PS C:\Users\Ivan\Downloads\cryptography-network-security-exam> python firewall_simulator.py
=====
Records Service : 192.168.1.70
Service Port : 8443
Staff Network : 192.168.1.0/24
Guest Network : 192.168.2.0/24
=====

[PASS] Test 1: Staff -> Records Service
Source : 192.168.1.50
Destination : 192.168.1.70/8443
Expected : ACCEPT
Actual : ACCEPT
Reason : Source is in the staff network and the service port is authorized

[PASS] Test 2: Guest -> Records Service
Source : 192.168.2.75
Destination : 192.168.1.70/8443
Expected : DROP
Actual : DROP
Reason : Source is in the simulated guest network

[PASS] Test 3: External host -> Records Service
Source : 203.0.113.50
Destination : 192.168.1.70/8443
Expected : DROP
Actual : DROP
Reason : Source is not in the authorized staff network

=====
All 3 Firewall tests passed.
PS C:\Users\Ivan\Downloads\cryptography-network-security-exam>
  
```

Figure 8: Firewall simulator output showing all three tests passing

7. Project Structure and Version Control

The work is managed in a public GitHub repository with a commit history that shows the project developing step by step. The repository is organised as follows:

```

cryptography-network-security-exam/
├── security_tool.py           encryption, decryption, SHA-256, error handling
├── student_record.txt        sample plaintext record
├── student_record.enc        encrypted record
├── student_record_decrypted.txt  decrypted output
├── risk_assessment.md        risk assessment
├── firewall_rules.txt        firewall rules and test results
├── firewall_simulator.py     firewall simulation
├── filter_tests.md           filtering test documentation
├── README.md                 project overview
├── report/                   LaTeX source and PDF report
  
```

The encryption key is deliberately excluded from the repository, in line with the principle that secrets must never be committed to version control.

8. Conclusion

This project identified the main risks to a student-records system and applied technical controls to address them. Fernet encryption protects the confidentiality of records, with the key stored outside the repository; SHA-256 checking detects unauthorised changes; robust error handling makes the tool predictable and safe to use; and a default-deny firewall design keeps guest and external traffic away from the records server.

Limitations and further work

- Fernet uses a single shared key, so key distribution and rotation are manual. A production system would use a key-management service.
- The firewall is a simulation; a real deployment would enforce these rules with tools such as iptables/nftables or a hardware firewall and would also include network segmentation using VLANs.
- Password weakness is addressed only as a recommendation; implementing multi-factor authentication and account lockout would strengthen the system further.
- Encryption of data in transit (for example TLS on port 8443) should be added so that records are protected during transfer as well as at rest.

References

Python Cryptographic Authority (n.d.) Fernet (symmetric encryption) – cryptography documentation. Available at: <https://cryptography.io/en/latest/fernet/>

National Institute of Standards and Technology (2015) FIPS PUB 180-4: Secure Hash Standard (SHS). Gaithersburg, MD: NIST.

Python Software Foundation (n.d.) ipaddress – IPv4/IPv6 manipulation library. Available at: <https://docs.python.org/3/library/ipaddress.html>

Internet Assigned Numbers Authority / RFC 5737 (2010) IPv4 Address Blocks Reserved for Documentation.